

React & Next.js

Jour 3 — Routing, Tests & Next.js



Sommaire — Jour 3

01 React Router — navigation SPA

03 useReducer — state complexe

05 Redux & Zustand — state global

07 Tests E2E avec Cypress

09 SSR, SSG, ISR & performances

02 Routes dynamiques & layouts

04 Context API — éviter le prop drilling

06 Tester avec Jest & RTL

08 Introduction à Next.js

10 Déploiement & comparatif frameworks

Rappel Jour 2

- Composants fonctionnels, props, `children`, réutilisabilité
- State : `useState`, immuabilité, lifting state up
- Formulaires contrôlés + validation
- Hooks : `useEffect`, `useRef`, `useMemo`, `useCallback`
- Custom hooks : `useFetch`, `useDebounce`
- TanStack Query : cache, mutations, invalidation

Aujourd'hui : architecture complète → routing, state global, tests et Next.js production

| 06 · React Router

Le Routing Côté Client — Concept

- Une **SPA** (Single Page Application) ne recharge jamais la page entière
- Le navigateur change l'URL → React affiche le bon composant
- Basé sur l'**API History** HTML5 (`pushState` , `popState`)

URL : /blog/react-hooks



React Router intercepte



Affiche `<BlogPost slug="react-hooks" />`
(sans rechargement de page)

Résultat : navigation *ultra-rapide* et transitions fluides



React Router — Installation

```
npm install react-router-dom
```

```
// src/main.jsx
import { BrowserRouter } from 'react-router-dom';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);

// BrowserRouter : utilise l'API History HTML5 (URLs propres)
// HashRouter      : utilise le hash (#/page) — pour hébergement statique sans config serveur
// MemoryRouter    : en mémoire — pour tests et React Native
```



React Router — Définir les Routes

```
// src/App.jsx
import { Routes, Route } from 'react-router-dom';
import HomePage      from './pages/HomePage';
import BlogPage      from './pages/BlogPage';
import PostPage      from './pages/PostPage';
import ProfilePage   from './pages/ProfilePage';
import NotFoundPage  from './pages/NotFoundPage';

function App() {
  return (
    <Routes>
      <Route path="/"          element={<HomePage />} />
      <Route path="/blog"      element={<BlogPage />} />
      <Route path="/blog/:slug" element={<PostPage />} />
      <Route path="/profile"   element={<ProfilePage />} />
      { /* Catch-all : tout ce qui ne matche pas */ }
      <Route path="*"         element={<NotFoundPage />} />
    </Routes>
  );
}
```



React Router — Navigation avec [Link](#)

```
import { Link, NavLink } from 'react-router-dom';

function Navigation() {
  return (
    <nav>
      { /* Link : navigation sans rechargement de page */ }
      <Link to="/">Accueil</Link>
      <Link to="/blog">Blog</Link>

      { /* NavLink : ajoute automatiquement la classe "active" */ }
      <NavLink
        to="/blog"
        className={({ isActive }) =>
          isActive ? 'nav-link active' : 'nav-link'
        }
      >
        Blog
      </NavLink>

      { /* Lien avec état (accessible via useLocation) */ }
      <Link to="/blog/react-hooks" state={{ from: 'homepage' }}>
        Article React Hooks
      </Link>
    </nav>
  );
}
```




React Router — Paramètres d'URL

```
import { useParams, useSearchParams, useLocation } from 'react-router-dom';

// Route définie : <Route path="/blog/:category/:slug" ... />
function PostPage() {
  // Paramètres de route → /blog/react/hooks-guide
  const { category, slug } = useParams();
  // category = "react", slug = "hooks-guide"

  // Query string → /blog?page=2&sort=date
  const [searchParams, setSearchParams] = useSearchParams();
  const page = searchParams.get('page') ?? '1';
  const sort = searchParams.get('sort') ?? 'date';

  // Changer les query params sans navigation complète
  const goToPage = (n) => setSearchParams({ page: n, sort });

  // Location complète
  const location = useLocation();
  // { pathname: '/blog/react', search: '?page=2', state: {...} }
```



React Router — Navigation Programmatique

```
import { useNavigate, Navigate } from 'react-router-dom';

function LoginForm() {
  const navigate = useNavigate();

  const handleLogin = async (credentials) => {
    try {
      await loginAPI(credentials);
      // Redirection après succès - replace: true empêche le retour arrière
      navigate('/dashboard', { replace: true });
    } catch (err) {
      setError(err.message);
    }
  };

  // Retour et avance dans l'historique
  const goBack = () => navigate(-1);
  const goForward = () => navigate(1);

  // Redirection déclarative (dans le JSX)
  if (isAuthenticated) {
    return <Navigate to="/dashboard" replace />;
  }

  return <form onSubmit={handleLogin}>...</form>;
}
```



React Router — Layouts Partagés avec Outlet

```
// src/layouts/MainLayout.jsx
import { Outlet } from 'react-router-dom';

function MainLayout() {
  return (
    <div className="app">
      <Header />
      <Sidebar />
      <main className="content">
        <Outlet />    {/* ← Route enfant rendue ici */}
      </main>
      <Footer />
    </div>
  );
}

// src/App.jsx - Routes imbriquées
<Routes>
  {/* Toutes ces routes partagent MainLayout */}
  <Route element={<MainLayout />}>
    <Route path="/" element={<HomePage />} />
    <Route path="/blog" element={<BlogPage />} />
    <Route path="/about" element={<AboutPage />} />
  </Route>
  {/* Cette route n'a pas de layout */}
  <Route path="/login" element={<LoginPage />} />
  <Route path="*" element={<NotFoundPage />} />
</Routes>
```



React Router — Route Protégée

```
// src/components/PrivateRoute.jsx
import { Navigate, Outlet } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth';

function PrivateRoute({ redirectTo = '/login' }) {
  const { isAuthenticated, isLoading } = useAuth();

  if (isLoading) return <Spinner />;

  // Redirige si non authentifié, sinon rend les enfants
  return isAuthenticated
    ? <Outlet />
    : <Navigate to={redirectTo} replace />;
}

// Utilisation dans App.jsx
<Routes>
  <Route path="/login" element={<LoginPage />} />

  {/* Routes protégées */}
  <Route element={<PrivateRoute />>
    <Route path="/dashboard" element={<Dashboard />} />
    <Route path="/profile" element={<ProfilePage />} />
    <Route path="/settings" element={<SettingsPage />} />
  </Route>
</Routes>
```



React Router — `useNavigate` & Scroll

```
// Scroll to top à chaque navigation
import { useEffect } from 'react';
import { useLocation } from 'react-router-dom';

function ScrollToTop() {
  const { pathname } = useLocation();
  useEffect(() => {
    window.scrollTo(0, 0);
  }, [pathname]);
  return null;
}

// Placer dans App.jsx ou dans le layout
function App() {
  return (
    <BrowserRouter>
      <ScrollToTop />
      <Routes>...</Routes>
    </BrowserRouter>
  );
}
```



Construire un **blog** multi-pages :

-  Home
-  `/blog/:slug`
-  Content
-  `NavLink`
-  Authentication
-  Deployment

Durée : 45 min



```
// src/App.jsx
export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route element={<MainLayout />} />
        <Route index element={<HomePage />} />
        <Route path="blog" element={<BlogPage />} />
        <Route path="blog/:slug" element={<PostPage />} />
        <Route path="about" element={<AboutPage />} />
      </Route>

      <Route element={<PrivateRoute />} />
      <Route path="profile" element={<ProfilePage />} />
    </Route>

    <Route path="login" element={<LoginPage />} />
    <Route path="*" element={<NotFoundPage />} />
  </Routes>
</BrowserRouter>
);
}
```

| 07 · Gestion de State Avancée



- `useState` devient **difficile à gérer** quand :
 - Plusieurs valeurs liées qui évoluent **ensemble**
 - Logique de mise à jour **complexe** ou conditionnelle
 - State suivant **dépend fortement** de l'état précédent
 - Besoin de **traçabilité** des transitions d'état

```
// ❌ useState : 4 variables séparées → mises à jour non atomiques
const [items, setItems] = useState([]);
const [loading, setLoading] = useState(false);
const [error, setError] = useState(null);
const [selected, setSelected] = useState(null);

// ✅ useReducer : un seul objet, transitions explicites
const [state, dispatch] = useReducer(reducer, {
  items: [], loading: false, error: null, selected: null,
});
```



```
// Un reducer = fonction PURE : (state, action) => newState
// Jamais de mutation directe, toujours retourner un nouveau state

function todoReducer(state, action) {
  switch (action.type) {
    case 'FETCH_START':
      return { ...state, loading: true, error: null };

    case 'FETCH_SUCCESS':
      return { ...state, loading: false, items: action.payload };

    case 'FETCH_ERROR':
      return { ...state, loading: false, error: action.payload };

    case 'ADD':
      return { ...state, items: [...state.items, action.payload] };

    case 'TOGGLE':
      return {
        ...state,
        items: state.items.map(t =>
          t.id === action.payload ? { ...t, done: !t.done } : t
        ),
      };

    case 'DELETE':
      return { ...state, items: state.items.filter(t => t.id !== action.payload) };

    default:
      return state;
  }
}
```



```
function TodoApp() {
  const [state, dispatch] = useReducer(todoReducer, {
    items: [], loading: false, error: null,
  });

  useEffect(() => {
    dispatch({ type: 'FETCH_START' });
    fetchTodos()
      .then(data => dispatch({ type: 'FETCH_SUCCESS', payload: data }))
      .catch(err => dispatch({ type: 'FETCH_ERROR', payload: err.message }));
  }, []);

  const addTodo = (title) => dispatch({ type: 'ADD', payload: { id: Date.now(), title, done: false } });
  const toggleTodo = (id) => dispatch({ type: 'TOGGLE', payload: id });
  const deleteTodo = (id) => dispatch({ type: 'DELETE', payload: id });

  const { items, loading, error } = state;
  if (loading) return <Spinner />;
  if (error) return <ErrorMessage msg={error} />;
  return <TodoList items={items} onToggle={toggleTodo} onDelete={deleteTodo} />;
}
```



Context API — Le Problème du Prop Drilling

```
// ✗ Prop drilling : theme traverse 4 niveaux inutilement
function App() {
  const [theme, setTheme] = useState('light');
  return <Layout theme={theme} setTheme={setTheme} />;
}
function Layout({ theme, setTheme }) {
  return <Sidebar theme={theme} setTheme={setTheme} />;
}
function Sidebar({ theme, setTheme }) {
  return <NavItem theme={theme} setTheme={setTheme} />;
}
function NavItem({ theme, setTheme }) {
  // Seul ce composant utilise theme/setTheme
  return (
    <button onClick={() => setTheme(t => t === 'light' ? 'dark' : 'light')}>
      {theme === 'light' ? '🌙' : '☀️'}
    </button>
  );
}
```



Context API — La Solution

```
import { createContext, useContext, useState } from 'react';

// 1. Créer le contexte
const ThemeContext = createContext(null);

// 2. Fournir via un Provider
export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () => setTheme(t => t === 'light' ? 'dark' : 'light');

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

// 3. Hook dédié (validation intégrée)
export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme doit être dans ThemeProvider');
  return ctx;
}

// 4. Utilisation — n'importe où dans l'arbre
function NavItem() {
  const { theme, toggleTheme } = useTheme();
  return <button onClick={toggleTheme}>{theme === 'light' ? '🌙' : '☀️'}</button>;
}
```

Context API — Auth Context Complet

```
// src/contexts/AuthContext.jsx
const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Restaurer la session au démarrage
    const token = localStorage.getItem('token');
    if (token) verifyToken(token).then(setUser).finally(() => setLoading(false));
    else setLoading(false);
  }, []);

  const login = async (credentials) => {
    const { user, token } = await loginAPI(credentials);
    localStorage.setItem('token', token);
    setUser(user);
  };

  const logout = () => {
    localStorage.removeItem('token');
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, logout, isAuthenticated: !!user, loading }}>
      {!loading && children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth hors du AuthProvider');
  return ctx;
};
```

Context — Limites & Bonnes Pratiques

✓ Bon usage

- **Thème** (light/dark)
- **Langue** (i18n/locale)
- **Utilisateur connecté**
- Données **rarement** mises à jour
- Configuration **globale**

⚠ Éviter pour...

- State **mis à jour fréquemment** (perf)
- **Cache de données serveur** → TanStack Query
- State complexe avec **beaucoup d'actions** → Redux/Zustand
- State **local** au composant → useState

⚠ *Tout composant consommant un Context se re-render quand la valeur change. Séparer les contextes par domaine (AuthContext, ThemeContext, CartContext).*



Zustand — Installation & Store

```
npm install zustand
```

```
// src/store/useCartStore.js
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

export const useCartStore = create(
  persist(
    (set, get) => ({
      items: [],

      addItem: (product) => set(state => {
        const existing = state.items.find(i => i.id === product.id);
        if (existing) {
          return { items: state.items.map(i =>
            i.id === product.id ? { ...i, qty: i.qty + 1 } : i
          )};
        }
        return { items: [...state.items, { ...product, qty: 1 }] };
      }),

      removeItem: (id) => set(s => ({ items: s.items.filter(i => i.id !== id) })),
      clearCart: () => set({ items: [] }),
      total: () => get().items.reduce((sum, i) => sum + i.price * i.qty, 0),
    }),
    { name: 'cart-storage' } // Persist dans localStorage
  )
);
```



```
// Pas de Provider nécessaire !

// Composant 1 : afficher le nombre d'articles
function CartIcon() {
  const count = useCartStore(s => s.items.length);
  return <span className="cart-badge">🛒 {count}</span>;
}

// Composant 2 : ajouter un produit
function ProductCard({ product }) {
  const addItem = useCartStore(s => s.addItem);
  return (
    <div>
      <h3>{product.name}</h3>
      <button onClick={() => addItem(product)}>
        Ajouter au panier
      </button>
    </div>
  );
}

// Composant 3 : le panier complet
function Cart() {
  const { items, removeItem, clearCart, total } = useCartStore();
  return <div>Total : {total().toFixed(2)} €</div>;
}
```

```
npm install @reduxjs/toolkit react-redux
```

```
// src/store/cartSlice.js
import { createSlice } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [], total: 0 },
  reducers: {
    addItem: (state, action) => {
      // Immer intégré : on peut "muter" l'état en toute sécurité !
      const existing = state.items.find(i => i.id === action.payload.id);
      if (existing) { existing.qty += 1; }
      else { state.items.push({ ...action.payload, qty: 1 }); }
      state.total += action.payload.price;
    },
    removeItem: (state, action) => {
      const idx = state.items.findIndex(i => i.id === action.payload);
      if (idx !== -1) {
        state.total -= state.items[idx].price * state.items[idx].qty;
        state.items.splice(idx, 1);
      }
    },
    clearCart: (state) => { state.items = []; state.total = 0; },
  },
});

export const { addItem, removeItem, clearCart } = cartSlice.actions;
export default cartSlice.reducer;
```

```
// src/store/index.js
import { configureStore } from '@reduxjs/toolkit';
import cartReducer from './cartSlice';
import themeReducer from './themeSlice';

export const store = configureStore({
  reducer: {
    cart: cartReducer,
    theme: themeReducer,
  },
});

// src/main.jsx
import { Provider } from 'react-redux';
import { store } from './store';

createRoot(document.getElementById('root')).render(
  <Provider store={store}>
    <App />
  </Provider>
);

// Redux DevTools Extension → indispensable en développement
// Permet de visualiser chaque action et voyager dans l'historique
```



Redux Toolkit — useSelector & useDispatch

```
import { useSelector, useDispatch } from 'react-redux';
import { addItem, removeItem } from '../store/cartSlice';

function CartButton({ product }) {
  const dispatch = useDispatch();
  const itemCount = useSelector(state => state.cart.items.length);
  const total = useSelector(state => state.cart.total);

  return (
    <div>
      <span>Panier : {itemCount} articles – {total.toFixed(2)} €</span>
      <button onClick={() => dispatch(addItem(product))}>
        + Ajouter
      </button>
    </div>
  );
}

// createSelector : mémorisation des sélecteurs complexes
import { createSelector } from '@reduxjs/toolkit';

const selectActiveItems = createSelector(
  state => state.cart.items,
  items => items.filter(i => i.qty > 0)
);
```



Context vs Zustand vs Redux Toolkit

Critère	Context API	Zustand	Redux Toolkit
Setup	Minimal	Minimal	Moyen
Provider requis	✓	✗	✓
Performance	⚠ Re-renders	✓ Sélecteurs	✓ Sélecteurs
DevTools	✗	✓ (partiel)	✓ Excellentes
Persistence	Manuel	✓ Middleware	✓ Redux Persist
Taille bundle	0	~1 ko	~11 ko
Courbe apprentissage	Faible	Faible	Moyenne
Idéal pour	Config globale	Apps moyennes	Grandes apps



Construire une mini app e-commerce :

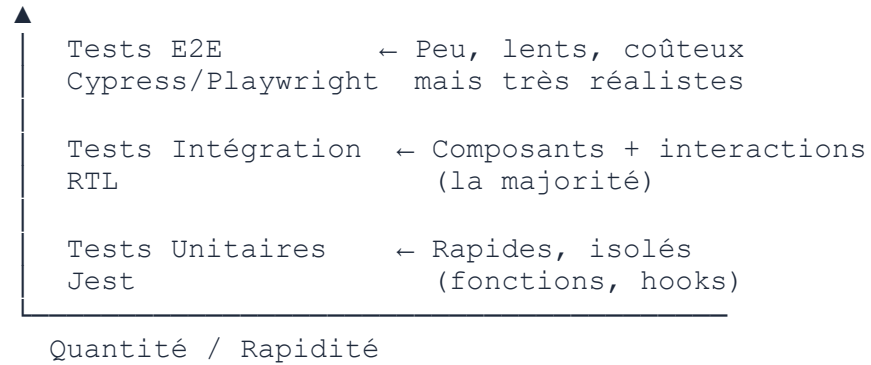
- 
- 
- 
- 
- 
- 

Zustand Redux Toolkit

Durée : 1h

| 08 · Tester une Application React

Stratégie de Tests — La Pyramide



- **Unitaires** : fonctions utilitaires, reducers, hooks isolés
- **Intégration** : composants avec leurs enfants et le DOM simulé
- **E2E** : flux utilisateur réels dans un vrai navigateur



Jest — Configuration Vite

```
npm install -D vitest jsdom @testing-library/react
npm install -D @testing-library/jest-dom @testing-library/user-event
```

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: ['./src/test/setup.js'],
  },
});
```

```
// src/test/setup.js
import '@testing-library/jest-dom';
```

```
// package.json
{ "scripts": { "test": "vitest", "test:ui": "vitest --ui" } }
```



Jest — Tests Unitaires Purs

```
// src/utils/formatPrice.test.js
import { formatPrice } from './formatPrice';

describe('formatPrice', () => {
  test('formate un entier correctement', () => {
    expect(formatPrice(19)).toBe('19,00 €');
  });

  test('formate un décimal correctement', () => {
    expect(formatPrice(19.9)).toBe('19,90 €');
  });

  test('retourne "Gratuit" pour un prix nul', () => {
    expect(formatPrice(0)).toBe('Gratuit');
  });

  test('lève une erreur pour un prix négatif', () => {
    expect(() => formatPrice(-5)).toThrow('Prix invalide');
  });
});

// src/utils/formatPrice.js
export function formatPrice(price) {
  if (price < 0) throw new Error('Prix invalide');
  if (price === 0) return 'Gratuit';
  return `${price.toFixed(2).replace('.', ',')} €`;
}
```



Jest — Tester un Reducer

```
// src/store/todoReducer.test.js
import todoReducer from './todoReducer';

const initial = { items: [], loading: false, error: null };

describe('todoReducer', () => {
  test('ADD : ajoute un todo', () => {
    const action = { type: 'ADD', payload: { id: 1, title: 'Test', done: false } };
    const state = todoReducer(initial, action);
    expect(state.items).toHaveLength(1);
    expect(state.items[0].title).toBe('Test');
  });

  test('TOGGLE : inverse done', () => {
    const withItem = { ...initial, items: [{ id: 1, title: 'T', done: false }] };
    const state = todoReducer(withItem, { type: 'TOGGLE', payload: 1 });
    expect(state.items[0].done).toBe(true);
  });

  test('DELETE : supprime le bon item', () => {
    const withItems = { ...initial, items: [{ id: 1, title: 'A', done: false }, { id: 2, title: 'B', done: false }] };
    const state = todoReducer(withItems, { type: 'DELETE', payload: 1 });
    expect(state.items).toHaveLength(1);
    expect(state.items[0].id).toBe(2);
  });

  test('action inconnue : retourne le state inchangé', () => {
    const state = todoReducer(initial, { type: 'UNKNOWN' });
    expect(state).toBe(initial);
  });
});
```



React Testing Library — Philosophie

"Test your components the way users interact with them"

— Kent C. Dodds

- Tester le **comportement visible**, pas l'implémentation interne
- Sélectionner par **rôle ARIA, label, texte** → accessibilité
- Ne jamais tester le state React directement → tester l'UI résultante

```
// ❌ Couplé à l'implémentation (fragile)
const btn = container.querySelector('.btn.btn--primary.btn--lg');

// ✅ Comme un vrai utilisateur (robuste)
const btn = screen.getByRole('button', { name: /ajouter au panier/i });
```



RTL — Hiérarchie des Sélecteurs

```
// Priorité recommandée (du plus au moins accessible)

// 1. Rôle ARIA — le plus robuste
screen.getByRole('button', { name: /envoyer/i })
screen.getByRole('textbox', { name: /email/i })
screen.getByRole('heading', { level: 1 })
screen.getByRole('checkbox', { name: /accepter/i })

// 2. Label des formulaires
screen.getByLabelText(/mot de passe/i)

// 3. Placeholder
screen.getByPlaceholderText(/rechercher.../i)

// 4. Texte visible
screen.getByText(/bienvenue sur notre site/i)

// 5. data-testid (dernier recours)
screen.getByTestId('dropdown-menu')

// Variants : getBy (erreur si absent), queryBy (null), findBy (async/Promise)
```



RTL — Tester un Composant

```
// src/components/Counter.test.jsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import Counter from './Counter';

describe('Counter', () => {
  test('affiche 0 par défaut', () => {
    render(<Counter />);
    expect(screen.getByText(/compteur : 0/i)).toBeInTheDocument();
  });

  test('incrémente au clic sur "+"', async () => {
    const user = userEvent.setup();
    render(<Counter />);

    await user.click(screen.getByRole('button', { name: '+' }));
    expect(screen.getByText(/compteur : 1/i)).toBeInTheDocument();

    await user.click(screen.getByRole('button', { name: '+' }));
    expect(screen.getByText(/compteur : 2/i)).toBeInTheDocument();
  });

  test('reset remet à 0', async () => {
    const user = userEvent.setup();
    render(<Counter initialValue={5} />);
    await user.click(screen.getByRole('button', { name: /reset/i }));
    expect(screen.getByText(/compteur : 0/i)).toBeInTheDocument();
  });
});
```



RTL — Tester un Formulaire

```
// src/components/LoginForm.test.jsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import LoginForm from './LoginForm';

describe('LoginForm', () => {
  test('affiche les erreurs si soumis vide', async () => {
    const user = userEvent.setup();
    render(<LoginForm onSubmit={vi.fn()} />);

    await user.click(screen.getByRole('button', { name: /connexion/i }));

    expect(screen.getByText(/email invalide/i)).toBeInTheDocument();
    expect(screen.getByText(/minimum 8 caractères/i)).toBeInTheDocument();
  });

  test('appelle onSubmit avec les bonnes valeurs', async () => {
    const handleSubmit = vi.fn();
    const user = userEvent.setup();
    render(<LoginForm onSubmit={handleSubmit} />);

    await user.type(screen.getByLabelText(/email/i), 'alice@example.com');
    await user.type(screen.getByLabelText(/mot de passe/i), 'password123');
    await user.click(screen.getByRole('button', { name: /connexion/i }));

    expect(handleSubmit).toHaveBeenCalledWith({
      email: 'alice@example.com',
      password: 'password123',
    });
  });
});
```



RTL — Tester l'Asynchrone

```
// src/components/UserProfile.test.jsx
import { render, screen, waitFor } from '@testing-library/react';
import UserProfile from './UserProfile';

// Mock de fetch global
global.fetch = vi.fn();

beforeEach(() => {
  fetch.mockResolvedValueOnce({
    ok: true,
    json: async () => ({ id: 1, name: 'Alice', email: 'alice@example.com' }),
  });
});

describe('UserProfile', () => {
  test('affiche le loader puis les données', async () => {
    render(<UserProfile userId={1} />);

    expect(screen.getByText(/chargement/i)).toBeInTheDocument();

    await waitFor(() => {
      expect(screen.getByText('Alice')).toBeInTheDocument();
      expect(screen.getByText('alice@example.com')).toBeInTheDocument();
    });

    expect(screen.queryByText(/chargement/i)).not.toBeInTheDocument();
  });
});
```




RTL — Tester un Custom Hook

```
// src/hooks/useCounter.test.js
import { renderHook, act } from '@testing-library/react';
import useCounter from './useCounter';

describe('useCounter', () => {
  test('valeur initiale par défaut = 0', () => {
    const { result } = renderHook(() => useCounter());
    expect(result.current.count).toBe(0);
  });

  test('initialise avec une valeur custom', () => {
    const { result } = renderHook(() => useCounter(10));
    expect(result.current.count).toBe(10);
  });

  test('increment fonctionne', () => {
    const { result } = renderHook(() => useCounter(0));
    act(() => result.current.increment());
    expect(result.current.count).toBe(1);
  });

  test('ne dépasse pas le maximum', () => {
    const { result } = renderHook(() => useCounter(9, { max: 10 }));
    act(() => result.current.increment());
    act(() => result.current.increment()); // Essaie de dépasser
    expect(result.current.count).toBe(10); // Bloqué
  });
});
```



Cypress — Tests End-to-End

```
npm install -D cypress
npx cypress open
# → Ouvre l'interface Cypress, choisir E2E Testing
```

```
// cypress/e2e/todo.cy.js
describe('Todo List', () => {
  beforeEach(() => {
    cy.visit('http://localhost:5173');
  });

  it('affiche le titre', () => {
    cy.contains('h1', 'Ma Todo List').should('be.visible');
  });

  it('ajoute une tâche', () => {
    cy.get('[data-cy=new-todo]').type('Apprendre Cypress');
    cy.get('[data-cy=add-btn]').click();
    cy.contains('Apprendre Cypress').should('be.visible');
  });

  it('supprime une tâche', () => {
    cy.get('[data-cy=todo-item]').first().find('[data-cy=delete]').click();
    cy.get('[data-cy=todo-item]').should('have.length', 0);
  });
});
```



Cypress — Commandes & Assertions

```
// cypress/e2e/auth.cy.js
describe('Authentification', () => {
  it('redirige vers /dashboard après login réussi', () => {
    cy.visit('/login');

    cy.get('[data-cy=email]').type('alice@example.com');
    cy.get('[data-cy=password]').type('password123');
    cy.get('[data-cy=submit]').click();

    cy.url().should('include', '/dashboard');
    cy.contains('Bienvenue, Alice').should('be.visible');
  });

  it('affiche une erreur si identifiants incorrects', () => {
    cy.visit('/login');
    cy.get('[data-cy=email]').type('bad@example.com');
    cy.get('[data-cy=password]').type('wrongpass');
    cy.get('[data-cy=submit]').click();
    cy.contains('Identifiants incorrects').should('be.visible');
    cy.url().should('include', '/login');
  });
});
```



Cypress — cy.intercept() — Mock API

```
// Intercepter les requêtes réseau pour des tests stables
describe('Catalogue produits', () => {
  beforeEach(() => {
    cy.intercept('GET', '/api/products*', {
      statusCode: 200,
      body: {
        products: [
          { id: 1, name: 'MacBook Pro', price: 2499 },
          { id: 2, name: 'iPhone 16', price: 1199 },
        ],
        total: 2,
      },
    }).as('getProducts'); // Alias pour les assertions
  });

  it('affiche les produits chargés depuis l\'API', () => {
    cy.visit('/products');
    cy.wait('@getProducts'); // Attendre que la requête soit interceptée
    cy.contains('MacBook Pro').should('be.visible');
    cy.contains('iPhone 16').should('be.visible');
  });
});
```



Écrire la suite de tests complète :

-  [reducer](#)
-  [TodoItem](#)
-  [formulaire d'ajout](#)
-  [Cypress E2E](#)

Durée : 45 min

| 09 · Next.js

Next.js — Positionnement

- **Framework** construit au-dessus de React (pas un remplaçant)
- Développé et maintenu par **Vercel** (depuis 2016)
- Tout ce que vous savez en React s'applique directement
- Ajoute : routing, SSR/SSG, optimisations, API Routes

React seul

SPA client-side
React Router
fetch manuel
Pas d'API
Config Webpack
Deploy manuel

Next.js

SSR + SSG + ISR + client
File-system routing (intégré)
Server Components (fetch natif)
API Routes / Route Handlers
Config intégrée + Turbopack
Vercel (1 clic)

Créer un Projet Next.js

```
npx create-next-app@latest mon-app-next
```

```
# Questions posées par l'assistant :
```

```
# ☒ TypeScript ?           → Oui (recommandé)
# ☒ ESLint ?             → Oui
# ☒ Tailwind CSS ?        → Oui
# ☒ App Router ?         → Oui (nouveau, recommandé)
# ☒ `src/` directory ?    → Non (optionnel)
# ☒ Turbopack ?          → Non (stable en 2025, optionnel)
# ☒ Import alias ?       → Non (optionnel)
```

```
cd mon-app-next
```

```
npm run dev
```

```
# ☒ http://localhost:3000
```


Structure — App Router (Next.js 13+)

```
mon-app/
├── app/                                # App Router
│   ├── layout.jsx                     # Layout racine (html, body)
│   ├── page.jsx                       # Route "/"
│   ├── globals.css
│   ├── about/
│   │   └── page.jsx                   # Route "/about"
│   ├── blog/
│   │   ├── page.jsx                  # Route "/blog"
│   │   └── [slug]/
│   │       └── page.jsx               # Route "/blog/:slug"
│   └── api/
│       └── posts/
│           └── route.js               # API "/api/posts"
├── components/                        # Composants partagés
├── public/                            # Fichiers statiques
└── next.config.js
```



```
// app/layout.jsx
import { Inter } from 'next/font/google';
import './globals.css';

// Téléchargement automatique au build + self-hosting
// Pas de requête réseau en production → pas de FOUT
const inter = Inter({ subsets: ['latin'], display: 'swap' });

export const metadata = {
  title: { default: 'Mon App', template: '%s | Mon App' },
  description: 'Description pour le SEO – apparaît dans Google',
  openGraph: { type: 'website', locale: 'fr_FR' },
};

export default function RootLayout({ children }) {
  return (
    <html lang="fr" className={inter.className}>
      <body>
        <Header />
        <main>{children}</main>
        <Footer />
      </body>
    </html>
  );
}
```

React Server Components (RSC)

```
// Server Component (DÉFAUT en App Router)
// Exécuté côté serveur uniquement
// ✅ Accès direct DB, fichiers, secrets
// ❌ Pas de useState, useEffect, event handlers

async function ProductList() {
  // Fetch direct côté serveur - aucun bundle client
  const products = await fetch('https://api.example.com/products')
    .then(r => r.json());

  return (
    <ul>
      {products.map(p => <li key={p.id}>{p.name}</li>)}
    </ul>
  );
}

// Client Component - doit être déclaré explicitement
'use client';

import { useState } from 'react';

function Counter() {
  const [n, setN] = useState(0); // ✅ Autorisé
  return <button onClick={() => setN(n + 1)}>{n}</button>;
}
```

Server vs Client Components

Server Component

- Défaut dans l'App Router
- Exécuté **côté serveur**
- Accès DB, FS, secrets d'env
- **Pas** de state, events, hooks
- Zéro JS envoyé au client
- Idéal pour les **données**

Client Component

- `'use client'` requis en haut
- Interactivité complète
- Tous les **hooks** disponibles
- Bundle JS envoyé au navigateur
- Rendu initial côté serveur aussi
- Minimiser leur nombre

Règle d'or : **pousser** `'use client'` *aussi loin que possible* dans l'arbre. Garder les parents en Server Components.



SSR — Server-Side Rendering

```
// app/products/page.jsx
// Page rendue à chaque requête → données toujours fraîches

async function getProducts() {
  const res = await fetch('https://api.example.com/products', {
    cache: 'no-store', // Pas de cache → rendu à la demande
  });
  if (!res.ok) throw new Error('Erreur API');
  return res.json();
}

export default async function ProductsPage() {
  const products = await getProducts(); // Await dans le composant !

  return (
    <section>
      <h1>Catalogue ({products.length} produits)</h1>
      <div className="grid">
        {products.map(p => <ProductCard key={p.id} product={p} />)}
      </div>
    </section>
  );
}

// ✅ Idéal pour : prix dynamiques, stock temps réel, pages personnalisées
```



SSG — Static Site Generation

```
// app/blog/[slug]/page.jsx
// Pages générées au BUILD → fichiers HTML statiques ultra-rapides

// Générer les paramètres au build
export async function generateStaticParams() {
  const posts = await fetch('https://api.example.com/posts')
    .then(r => r.json());

  return posts.map(post => ({ slug: post.slug }));
}

// Métadonnées dynamiques (SEO)
export async function generateMetadata({ params }) {
  const post = await getPost((await params).slug);
  return { title: post.title, description: post.excerpt };
}

export default async function BlogPost({ params }) {
  const { slug } = await params;
  const post = await fetch(`https://api.example.com/posts/${slug}`)
    .then(r => r.json());

  return <article><h1>{post.title}</h1><p>{post.content}</p></article>;
}

// ✅ Idéal pour : blog, docs, landing pages, contenu marketing
```



ISR — Incremental Static Regeneration

```
// Meilleur des deux mondes : statique + actualisable

async function getPosts() {
  const res = await fetch('https://api.example.com/posts', {
    next: { revalidate: 300 }, // Régénérer toutes les 5 minutes max
  });
  return res.json();
}

export default async function BlogPage() {
  const posts = await getPosts();
  return (
    <ul>
      {posts.map(p => <li key={p.id}><a href={` /blog/${p.slug} `}>{p.title}</a></li>)}
    </ul>
  );
}

// Fonctionnement ISR :
// 1. Requête → page statique servie immédiatement (rapide)
// 2. En parallèle : Next.js vérifie si revalidate est dépassé
// 3. Si oui → régénération silencieuse en arrière-plan
// 4. Prochaine requête → page fraîche
```

SSR vs SSG vs ISR vs CSR

Mode	Quand généré	Cache	Idéal pour	Perf
SSG	Au build	Permanent	Blog, docs, marketing	⚡⚡⚡
ISR	Build + revalidate	Temps limité	Catalogue, actualités	⚡⚡
SSR	Chaque requête	Aucun	Dashboard, contenu perso	⚡
CSR	Navigateur	Variable	Admin auth, SPA	⚡ (après chargement)



Optimisation Images — [next/image](#)

```
import Image from 'next/image';

function HeroSection() {
  return (
    <section>
      { /* Optimisations automatiques : WebP/AVIF, lazy, srcset, CLS */ }
      <Image
        src="/hero.jpg"
        alt="Image hero de la page d'accueil"
        width={1200}
        height={600}
        priority           // Chargement prioritaire (above-the-fold)
        placeholder="blur"
        blurDataURL="data:image/jpeg;base64,..."
      />
    </section>
  );
}

function ProductCard({ product }) {
  return (
    <Image
      src={product.imageUrl}
      alt={product.name}
      fill           // Remplit le conteneur parent (position: relative requis)
      sizes="(max-width: 768px) 100vw, 300px"
      className="object-cover"
    />
  );
}
```

API Routes — Route Handlers

```
// app/api/posts/route.js

import { NextResponse } from 'next/server';

// GET /api/posts
export async function GET(request) {
  const { searchParams } = new URL(request.url);
  const page = parseInt(searchParams.get('page') ?? '1');
  const limit = parseInt(searchParams.get('limit') ?? '10');

  const posts = await db.posts.findMany({
    skip: (page - 1) * limit,
    take: limit,
    orderBy: { createdAt: 'desc' },
  });

  return NextResponse.json({ posts, page, total: posts.length });
}

// POST /api/posts
export async function POST(request) {
  const body = await request.json();
  if (!body.title?.trim()) {
    return NextResponse.json({ error: 'Titre requis' }, { status: 400 });
  }
  const post = await db.posts.create({ data: body });
  return NextResponse.json(post, { status: 201 });
}
```

API Routes — Route Dynamique

```
// app/api/posts/[id]/route.js

export async function GET(request, { params }) {
  const { id } = await params;
  const post = await db.posts.findUnique({ where: { id: parseInt(id) } });

  if (!post) {
    return NextResponse.json({ error: 'Article introuvable' }, { status: 404 });
  }
  return NextResponse.json(post);
}

export async function PUT(request, { params }) {
  const { id } = await params;
  const body = await request.json();
  const post = await db.posts.update({
    where: { id: parseInt(id) },
    data: body,
  });
  return NextResponse.json(post);
}

export async function DELETE(request, { params }) {
  const { id } = await params;
  await db.posts.delete({ where: { id: parseInt(id) } });
  return new Response(null, { status: 204 });
}
```



Middleware Next.js

```
// middleware.js (racine du projet)
import { NextResponse } from 'next/server';

export async function middleware(request) {
  const { pathname } = request.nextUrl;
  const token = request.cookies.get('auth-token')?.value;

  // Protection des routes authentifiées
  if (pathname.startsWith('/dashboard') || pathname.startsWith('/profile')) {
    if (!token) {
      const loginUrl = new URL('/login', request.url);
      loginUrl.searchParams.set('from', pathname); // Rediriger après login
      return NextResponse.redirect(loginUrl);
    }
  }

  // Redirection d'anciennes URLs
  if (pathname === '/blog/old-slug') {
    return NextResponse.redirect(new URL('/blog/new-slug', request.url));
  }

  return NextResponse.next();
}

// Appliquer uniquement sur certains chemins (performance)
export const config = {
  matcher: ['/dashboard/:path*', '/profile/:path*', '/blog/old-slug'],
};
```

Variables d'Environnement Next.js

```
# .env.local (non committé en git)
DATABASE_URL=postgresql://...
JWT_SECRET=super-secret-key-never-expose

# Accessible côté serveur uniquement (sécurisé)
API_SECRET_KEY=private-key

# Accessible côté client (préfixe NEXT_PUBLIC_)
NEXT_PUBLIC_API_URL=https://api.example.com
NEXT_PUBLIC_ANALYTICS_ID=UA-XXXXX
```

```
// Serveur uniquement (dans un Server Component ou Route Handler)
const secret = process.env.JWT_SECRET; // ✓

// Client + Serveur (préfixe NEXT_PUBLIC_)
const apiUrl = process.env.NEXT_PUBLIC_API_URL; // ✓

// ✗ JAMAIS côté client sans NEXT_PUBLIC_
const secret = process.env.JWT_SECRET; // undefined en client !
```



Déploiement sur Vercel

- 1 Pousser le code sur GitHub (`git push origin main`)
- 2 Aller sur vercel.com → « Add New Project »
- 3 Importer le repository GitHub
- 4 Configurer les variables d'environnement (Settings → Environment Variables)
- 5 Cliquer « Deploy » → URL automatique générée
- 6 Domaine custom → Settings → Domains → Ajouter votre domaine

Déploiement — `next build` Output

```
npm run build
```

```
# Output de la commande :
```

Route (app)	Size	First Load JS
o /	5.2 kB	87.4 kB
o /about	1.1 kB	83.3 kB
• /blog	2.4 kB	84.6 kB
• /blog/[slug]	3.8 kB	86.0 kB
f /api/posts	0 B	0 B

```
o Static — généré au build
```

```
• ISR — revalidation périodique
```

```
f Dynamic — généré à la demande (SSR / API)
```

Analyse des Performances

```
# Next.js Bundle Analyzer
npm install -D @next/bundle-analyzer

# next.config.js
const withBundleAnalyzer = require('@next/bundle-analyzer')({
  enabled: process.env.ANALYZE === 'true',
});
module.exports = withBundleAnalyzer({});

# Lancer l'analyse
ANALYZE=true npm run build
# → Ouvre un treemap visuel du bundle
```

Core Web Vitals à surveiller :

- **LCP** (Largest Contentful Paint) → < 2.5s
- **FID / INP** (Interaction to Next Paint) → < 200ms
- **CLS** (Cumulative Layout Shift) → < 0.1



Migrer le blog React vers **Next.js** :

-  `create-next-app`
- 
- 
- 
-  `generateStaticParams`
-  `next/image`
-  `/api/posts`
-  `Vercel`



```
app/
├── layout.jsx           # Layout avec Header + Footer
├── page.jsx             # Accueil → articles récents (ISR 1h)
├── about/
│   └── page.jsx         # Page statique (SSG)
├── blog/
│   ├── page.jsx        # Liste (ISR 5min)
│   └── [slug]/
│       ├── page.jsx     # Article (SSG + generateStaticParams)
│       └── loading.jsx  # UI de chargement (Suspense)
├── api/
│   └── posts/
│       ├── route.js     # GET, POST /api/posts
│       └── [id]/
│           └── route.js  # GET, PUT, DELETE /api/posts/:id
└── not-found.jsx       # Page 404 custom
```



```
# 1. Commit et push
git add .
git commit -m "Migration Next.js complète"
git push origin main

# 2. Vercel CLI (optionnel)
npm install -g vercel
vercel login
vercel --prod

# 3. Variables d'environnement
# Dans Vercel Dashboard → Settings → Environment Variables
# NEXT_PUBLIC_API_URL = https://api.example.com
# DATABASE_URL = postgresql://...

# 4. Vérifications post-déploiement
# → Lighthouse score (viser 90+ en Performance)
# → Vérifier les routes SSG dans le build output
# → Tester les API routes avec Postman/Thunder Client
```

| 10 · Comparatif des Frameworks React

Les Frameworks React Production

▲ Next.js

Référence pour les apps hybrides SSR/SSG. Écosystème Vercel, App Router, RSC. Fort en SEO et performance.

Remix / React Router

Web standards, loaders/actions, gestion des erreurs intégrée. Idéal pour apps orientées formulaires et mutations.

TanStack Start

Solution légère et progressive. Router typé, isomorphe. Idéal pour SPAs complexes et apps TypeScript.

Gatsby

Spécialisé JAMstack et contenus statiques. CMS headless, GraphQL. Fort pour sites marketing et e-commerce statique.



Comparatif Complet des Frameworks

Critère	Next.js	Remix	TanStack Start	Gatsby
Rendu	SSR/SSG/ISR/RSC	SSR/CSR	SSR/CSR	SSG/DSG
Routing	File-system	File-system	Typed	File-system
Data fetching	fetch + RSC	Loaders/Actions	Typed loaders	GraphQL
TypeScript	✅ Fort	✅ Fort	✅ Natif	✅ Fort
Écosystème	★★★★★	★★★★★	★★★★	★★★★
Deploy	Vercel / Partout	Partout	Partout	Netlify/Vercel
Popularité 2025	🏆 1er	🏆 2e	🏆 3e	Niche
Courbe apprentissage	Moyenne	Moyenne	Faible	Faible
Idéal pour	Apps hybrides	Forms/Mutations	SPAs typées	Sites statiques

Comment Choisir ?

- **Next.js** → Choix par défaut pour 90% des projets. Grande équipe, marketing, e-commerce, dashboards, apps hybrides.
- **Remix** → Si votre app est très orientée mutations (formulaires, CRUD), que vous aimez les web standards et les loaders/actions.
- **TanStack Start** → Si TypeScript strict est une priorité et que vous préférez une solution légère et progressive.
- **Gatsby** → Uniquement pour des sites majoritairement statiques avec une source de contenu CMS headless (Contentful, Sanity, etc.).

*En 2025 : commencez avec **Next.js**. Changez de framework seulement si un besoin spécifique l'exige.*



Quiz Jour 3 — Question 1

Quelle est la bonne façon de définir une route dynamique `/blog/:slug` dans React Router ?

- A) `<Route path="/blog/[slug]" element={<PostPage />} />`
- B) `<Route path="/blog/:slug" element={<PostPage />} />` ✓
- C) `<Route path="/blog/{slug}" element={<PostPage />} />`
- D) `<Route path="/blog/$slug" element={<PostPage />} />`

React Router v6 utilise la syntaxe `:paramName` pour les paramètres dynamiques, récupérés avec `useParams()`.



Quiz Jour 3 — Question 2

Quelle est la différence entre SSG et SSR dans Next.js ?

- A) SSG génère les pages **au moment de la requête**, SSR au build
- B) SSG génère les pages **au build**, SSR à chaque requête ☒
- C) SSG est plus lent que SSR à servir
- D) Il n'y a aucune différence en Next.js 14+

SSG (Static Site Generation) génère des fichiers HTML statiques au `next build` → ultra-rapides à servir. SSR (Server-Side Rendering) génère la page à chaque requête → données toujours fraîches.



Quiz Jour 3 — Question 3

Que faut-il ajouter en haut d'un fichier pour créer un Client Component dans Next.js ?

- A) `import 'react-client';`
- B) `// @client`
- C) `'use client';` 
- D) `export const runtime = 'client';`

La directive `'use client'` en haut du fichier (avant tout import) indique à Next.js que ce composant s'exécute côté navigateur et peut utiliser `useState`, `useEffect` et les event handlers.



Quiz Jour 3 — Question 4

Dans React Testing Library, quel sélecteur est recommandé en priorité ?

- A) `getByClassName`
- B) `getByTestId`
- C) `getByRole` ✓
- D) `getBySelector`

`getByRole` est prioritaire car il teste l'accessibilité réelle du composant. Un bouton trouvé par `getByRole('button', { name: /submit/i })` sera aussi trouvé par un lecteur d'écran → tests robustes et accessibles.



Quiz Jour 3 — Question 5

Quelle est la différence entre `useReducer` et `useState` ?

- A) `useReducer` est réservé aux class components
- B) `useState` ne peut gérer que des types primitifs
- C) `useReducer` est adapté à un state complexe avec plusieurs sous-valeurs liées et des transitions explicites ☒
- D) Il n'y a aucune différence, les deux sont identiques

`useReducer` centralise la logique de mise à jour dans un reducer pur, facilite les tests, et est idéal quand plusieurs valeurs évoluent ensemble ou quand les transitions doivent être traçables.

Récapitulatif Jour 3

-  **React Router** : `BrowserRouter`, `Routes`, `Route`, `Link`, `NavLink`, `useParams`, `useNavigate`, layouts avec `Outlet`
-  **useReducer** : state complexe, reducer pur, transitions explicites
-  **Context API** : distribution globale sans prop drilling, custom provider
-  **Zustand** : state global simple, sans Provider, persist middleware
-  **Redux Toolkit** : slice, configureStore, useSelector, useDispatch
-  **Jest + RTL** : tests unitaires, composants, formulaires, hooks, async
-  **Cypress** : E2E, intercept API, assertions sur le DOM réel
-  **Next.js** : App Router, RSC, SSR/SSG/ISR, API Routes, Middleware
-  **Déploiement** : Vercel, variables d'environnement, Core Web Vitals

Ressources pour Continuer

- react.dev — Documentation officielle React (avec hooks)
- nextjs.org/docs — Documentation Next.js App Router
- tanstack.com — TanStack Query, Router, Table
- testing-library.com — RTL docs et bonnes pratiques
- docs.cypress.io — Cypress guides et exemples
- zustand-demo.pmnd.rs — Zustand docs et exemples
- redux-toolkit.js.org — Redux Toolkit et RTK Query
- web.dev — Core Web Vitals et performance web

Projets suggérés pour consolider : clone de Hacker News (SSR + TanStack Query), blog MDX complet (SSG), dashboard e-commerce (Zustand + Next.js)

Merci pour votre attention ! 🎉

contact@formation.fr